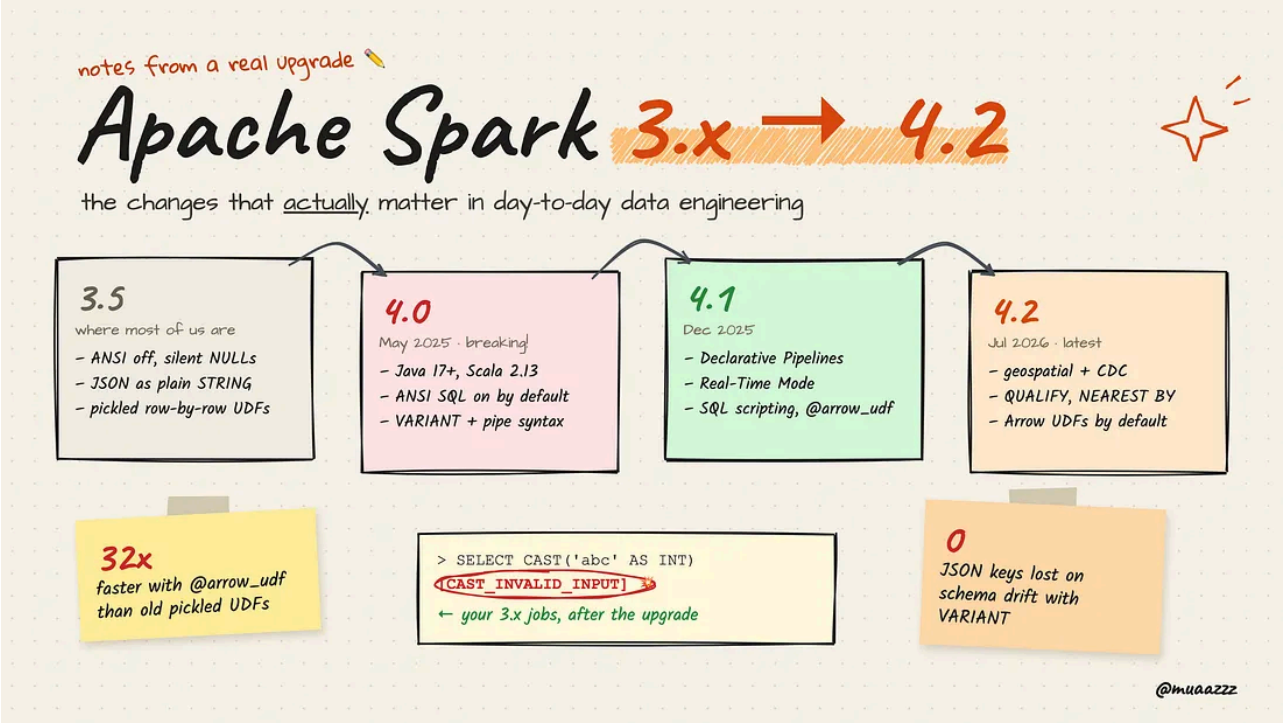
Data Science | Data Engineering | Spark | Big Data | Artificial Intelligence

Apache Spark 4.0 to 4.2: What's New, What's Changed, and What Data Engineers Should Know

Muaaz Follow 12 min read · 5 days ago

52 2 1



In modern data engineering, Apache Spark sits at the center of almost every Big data pipeline. For a long time most of us were comfortable on Spark 3.x,

but within roughly a year the community shipped three big releases: **Spark 4.0 (May 2025)**, **Spark 4.1 (December 2025)** and **Spark 4.2 (July 14, 2026)**

Each of these releases comes with many of changes. In this blog we will not go through all of them. Instead, we will focus on the **major changes that we actually use in our day-to-day work** and that have the most real use cases.

The Big Picture: 3.5 vs 4.0 vs 4.1 vs 4.2

- Spark 3.5.x: the final minor release line before Spark 4.0, introducing Spark Connect GA, Python UDTFs, SQL IDENTIFIER clause, HyperLogLog sketches, DeepSpeed integration, streaming improvements, and enhanced PySpark testing and error handling.
- **Spark 4.0**: the major release with **breaking changes**. Java 17+, Scala 2.13 only, ANSI SQL on by default, VARIANT type, SQL pipe syntax, Python Data Source API, mature Spark Connect.
- **Spark 4.1**: a feature release. **Spark Declarative Pipelines**, Real-Time Mode in streaming, SQL scripting GA, recursive CTEs, VARIANT GA with shredding, `@arrow_udf`, Spark ML on Connect GA.
- **Spark 4.2**: the latest feature release. Geospatial types, CDC with the `CHANGES` clause, `QUALIFY`, Arrow-optimized Python UDFs **on by default**, `NEAREST BY` joins, metric views, Java 25 support and a web UI with dark mode.

We will explore the changes section by section, following the same structure as Apache Spark itself, covering Spark Core, Build & Dependencies, Spark SQL, PySpark APIs, Spark Connect, Structured Streaming, Declarative Pipelines, and Deployment / Cluster Managers.

Build & Dependencies

Before exploring the new features, it's worth looking at the changes under the hood. Spark 4.x introduces newer runtime requirements and dependency upgrades that can affect existing applications, custom connectors, and deployment environments.

Area	Spark 3.5.8	Spark 4.0	Spark
Java	Java 8, 11, or 17	Java 17+	Java 1
Scala	2.12 or 2.13	2.13 only	2.13 o
Python	3.8+	3.9+	3.10+
Hadoop	3.x-compatible distributions	Hadoop 3-based distributions	Check
PyArrow	11+	11+	15+

An upgrade is not simply a matter of changing the Spark version in your environment. You need to check whether your custom JARs, third-party connectors, Python packages, and deployment images remain compatible. In particular, moving to Scala 2.13 can require replacing dependencies built for Scala 2.12.

Before upgrading, audit your dependencies and test representative workloads in the target environment. These checks can reveal compatibility issues before they reach production.

Spark SQL

Spark SQL gets the largest number of changes across these three releases.

VARIANT Data Type [4.0, GA in 4.1]

This is in my opinion the most useful new feature for day-to-day work. Most raw data we ingest (events, clickstream, API responses, IoT messages) arrives as JSON whose keys change over time.

On Spark 3.5 we had two options, and neither was great:

- Keep JSON as `STRING` and call `get_json_object`, which **re-parses the text for every field in every query.**
- Use `from_json(payload, schema)`, which **only knows the keys in your schema.**

Spark 4.0 adds the `VARIANT` type. `parse_json()` converts JSON **once** into a compact binary format that keeps every key **and its type.**

```
json_data = [
    ('{"user_id": 101, "name": "Ali", "city": "Lahore"}',),
    ('{"user_id": 102, "name": "Sara", "device": "iPhone"}',),
    ('{"user_id": 103, "name": "Ahmed", "premium": true}',),
    ('{"user_id": 104, "name": "Usman", "age": 25}',),
]

df = spark.createDataFrame(json_data, ["json_string"])
variant_df = df.select(F.parse_json("json_string").alias("payload"))
variant_df.printSchema()
```

```
root
 |-- payload: variant (nullable = true)
```

Then we read the fields with `variant_get` (or the shorthand `payload:city::string`):

```
variant_df.createOrReplaceTempView("variant_events")

spark.sql("""
    SELECT
        variant_get(payload, '$.user_id', 'INT')     AS user_id,
        variant_get(payload, '$.name', 'STRING')     AS name,
```

```
        variant_get(payload, '$.city', 'STRING')     AS city,
        variant_get(payload, '$.device', 'STRING')   AS device,
        variant_get(payload, '$.premium', 'BOOLEAN') AS premium,
        variant_get(payload, '$.age', 'INT')         AS age
    FROM variant_events
""").show()
```

```
+-------+-----+------+------+-------+----+
|user_id|name |city  |device|premium|age |
+-------+-----+------+------+-------+----+
|101    |Ali  |Lahore|NULL  |NULL   |NULL|
|102    |Sara |NULL  |iPhone|NULL   |NULL|
|103    |Ahmed|NULL  |NULL  |true   |NULL|
|104    |Usman|NULL  |NULL  |NULL   |25  |
+-------+-----+------+------+-------+----+
```

SQL Pipe Syntax and Session Variables [4.0]

Pipe syntax lets us write a query **top to bottom in the order it runs**, instead of nesting `SELECT ... FROM (SELECT ...)`. Session variables (`DECLARE VARIABLE` / `SET VAR`) let us hold values inside SQL, where on 3.5 we had to build the SQL string in Python

```
DECLARE OR REPLACE VARIABLE min_amount DOUBLE DEFAULT 100;
SET VAR min_amount = 250;

FROM orders
|> WHERE amount > min_amount
|> EXTEND get_json_object(payload, '$.city') AS city
|> AGGREGATE COUNT(*) AS orders, ROUND(SUM(amount), 2) AS revenue GROUP BY city
|> ORDER BY revenue DESC
```

There is no speed difference here; both forms compile to the same plan. It is purely about readability. Spark 4.0 also adds `EXECUTE IMMEDIATE` for dynamic

SQL and **string collations** for case- and accent-insensitive comparisons (4.2 extends collation support and adds `SHOW COLLATIONS`).

Recursive CTEs [4.1]

Hierarchies such as org charts, bill of materials or category trees can finally be walked in plain SQL:

```
WITH RECURSIVE tree(id, name, parent, path, depth) AS (
    SELECT id, name, parent, name, 0 FROM cats WHERE parent IS NULL
    UNION ALL
    SELECT c.id, c.name, c.parent, t.path || ' > ' || c.name, t.depth + 1
    FROM cats c JOIN tree t ON c.parent = t.id
)
SELECT depth, path FROM tree ORDER BY path
```

```
+-----+----------------------+
|depth|path                  |
+-----+----------------------+
|0    |All                   |
|1    |All > Laptops         |
|2    |All > Laptops > Gaming|
|1    |All > Phones          |
|2    |All > Phones > Android|
|2    |All > Phones > iOS    |
+-----+----------------------+
```

SQL Scripting [Preview in 4.0, GA in 4.1]

scripting brings `BEGIN ... END` blocks with variables, `IF`, `WHILE`, `FOR`, `LOOP` and **exception handlers**, all sent as a single `spark.sql(...)` call. **[4.2]** adds **cursors** inside scripts.

```
script = """
BEGIN
  DECLARE total DOUBLE DEFAULT 0;
  DECLARE status STRING DEFAULT 'ok';
```

```
  FOR c AS (SELECT DISTINCT variant_get(payload, '$.city', 'string') AS city FRO
    SET total = total + (SELECT sum(amount) FROM ev_var WHERE variant_get(payloa
  END FOR;
  BEGIN
    DECLARE EXIT HANDLER FOR DIVIDE_BY_ZERO SET status = 'caught DIVIDE_BY_ZERO,
    SELECT 1 / 0;
  END;
  SELECT format_number(total, 2) AS revenue_all_cities, status;
END"""
spark.sql(script).show(truncate=False)
```

Sketch Functions [4.1]

`approx_top_k`, KLL quantiles and Theta distinct counts. The key benefit is that sketches can be **stored and merged later**, for example daily sketches rolled up into a monthly answer.

```
SELECT approx_top_k(city, 3)                                          AS top
       kll_sketch_get_quantile_double(kll_sketch_agg_double(amount), 0.5) AS med
       theta_sketch_estimate(theta_sketch_agg(customer_id))           AS dis
FROM orders_v
```

QUALIFY, time_bucket and More [4.2]

`QUALIFY` filters on window functions **without a subquery**. The classic "latest or top-N row per group" becomes one clean query:

```
SELECT
    employee_id,
    department,
    salary,
    DENSE_RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS salary_rank
```

```
FROM employees
QUALIFY salary_rank <= 2;Other SQL additions in 4.2 that are worth knowing:
```

- `time_bucket` for fixed-size time buckets in dashboards.
- `max_by(x, y, k)` returns the top-k values, and `collect_list(x) IGNORE NULLS`.
- `TABLESAMPLE SYSTEM` for block-level sampling.
- `SET PATH` / `CURRENT_PATH()` to control function resolution.
- `INSERT ... WITH SCHEMA EVOLUTION`.
- **Metric views:** `CREATE VIEW ... WITH METRICS` to define reusable business metrics once.
- **`NEAREST BY` joins** for top-K nearest-neighbour queries.
- **Vector functions:** `vector_cosine_similarity`, `vector_l2_distance`, `vector_inner_product`, `vector_norm` and more, which make simple embedding similarity possible in plain SQL.

```
SELECT vector_cosine_similarity(array(1.0F, 0.0F), array(1.0F, 1.0F)) AS cos_sim
       vector_l2_distance(array(0.0F, 0.0F), array(3.0F, 4.0F))       AS l2
```

Geospatial Types [4.2]

Spark 4.2 adds native `GEOMETRY` and `GEOGRAPHY` types, `ST_*` functions and Parquet read/write for them.

```
SELECT typeof(st_geomfromwkb(<wkb>))                    AS geom_type,   -- geom
       typeof(st_geogfromwkb(<wkb>))                    AS geog_type,   -- geog
```

```
    st_srid(st_setsrid(st_geomfromwkb(<wkb>), 4326))  AS srid
```

> Note: in the 4.2.0 release only the WKB-based `st_*` functions shipped. WKT functions like `st_astext` / `st_geomfromtext` were not available.

Change Data Capture with the CHANGES Clause [4.2]

Spark 4.2 adds a SQL `CHANGES` clause and a matching DataFrame API (`spark.read.changes(...)`) for reading change data from tables whose catalog supports it, plus **DSv2 transactions** so connectors can commit several operations atomically.

PySpark APIs

Python Data Source API [4.0]

On Spark 3.5, writing a custom connector meant writing **Scala or Java** against DataSource V2. Spark 4.0 lets us write readers and writers in **pure Python**. [4.1] adds filter pushdown for Python sources, and [4.2] adds admission control and `AvailableNow` for Python streaming sources.

Here is a small connector that reads a REST API as a Spark DataFrame:

```
from pyspark.sql.datasource import DataSource, DataSourceReader
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

class PostsAPIDataSource(DataSource):
    """Read posts from a REST API as a Spark DataFrame."""
    @classmethod
    def name(cls):
        return "posts_api"
    def schema(self):
        return StructType([
            StructField("id", IntegerType(), True),
            StructField("user_id", IntegerType(), True),
            StructField("title", StringType(), True),
```

```
            StructField("body", StringType(), True),
        ])
    def reader(self, schema):
        return PostsAPIReader(self.options)

class PostsAPIReader(DataSourceReader):
    def __init__(self, options):
        self.options = options
    def read(self, partition):
        import requests
        url = self.options.get("url", "https://jsonplaceholder.typicode.com/post
        response = requests.get(url, timeout=int(self.options.get("timeout", "30
        response.raise_for_status()
        for record in response.json():
            yield (record.get("id"), record.get("userId"), record.get("title"),
```

Register it once, and from then on it behaves like any built-in format:

```
spark.dataSource.register(PostsAPIDataSource)

posts_df = spark.read.format("posts_api").load()
print("Total records:", posts_df.count())   # Total records: 100
```

Python UDFs: Arrow Everywhere [4.1, default in 4.2]

- [4.0] A unified **UDF profiler:** set `spark.sql.pyspark.udf.profiler = perf` (or `memory`) and call `spark.profile.show()` to see where the time goes inside your UDF.
- [4.1] `@arrow_udf`: the function receives and returns `pyarrow.Array` directly, with no pandas conversion in between.
- [4.2] `spark.sql.execution.pythonUDF.arrow.enabled = true` **by default**, so a plain `@udf` now moves data in Arrow batches instead of pickling row by row. Use `useArrow=False` to get the old behaviour.

```
import pyarrow as pa, pyarrow.compute as pc
from pyspark.sql.functions import udf, pandas_udf, arrow_udf

@udf("double", useArrow=False)           # 3.5 / 4.0 default: pickled
def tax_pickled(x): return round(x * 1.17, 2)

@udf("double")                           # 4.2 default: Arrow-batched
def tax_default(x): return round(x * 1.17, 2)

@pandas_udf("double")
def tax_pandas(s: pd.Series) -> pd.Series: return (s * 1.17).round(2)

@arrow_udf("double")                     # 4.1: pyarrow in, pyarrow out
def tax_arrow(a: pa.Array) -> pa.Array: return pc.round(pc.multiply(a, 1.17), 2)
```

Polymorphic Python UDTFs [4.0]

On 3.5, a Python UDTF had a **fixed** `returnType`. Whatever you passed in, it came out as the same type:

```
@udtf(returnType="value: string, python_type: string")
class FixedInspector:
    def eval(self, value):
        yield str(value), type(value).__name__
```

Spark 4.0 adds a static `analyze()` method that decides the **output schema at query planning time** based on the input:

```
from pyspark.sql.functions import udtf
from pyspark.sql.types import StructType, StringType
from pyspark.sql.udtf import AnalyzeArgument, AnalyzeResult

@udtf
class PolymorphicInspector:
    @staticmethod
    def analyze(value: AnalyzeArgument) -> AnalyzeResult:
```

```
            schema = StructType()
            schema.add("value", value.dataType)       # keep the caller's type
            schema.add("python_type", StringType())
            return AnalyzeResult(schema=schema)
        def eval(self, value):
            yield value, type(value).__name__
spark.udtf.register("poly_inspector", PolymorphicInspector)
spark.sql("SELECT * FROM poly_inspector(42)").printSchema()
```

```
root
 |-- value: integer (nullable = true)
 |-- python_type: string (nullable = true)
```

Pass `'hello'` and `value` becomes a `string`; pass `CAST(3.14 AS DOUBLE)` and it becomes a `double`. It even works with arrays, maps, nested structs and VARIANT. **[4.1]** adds vectorized `@arrow_udtf`, and **[4.2]** turns Arrow on by default for UDTFs too.

Use cases: generic explode / flatten / split functions that keep the caller's column type, and table functions whose output columns depend on an argument.

Other PySpark Improvements

- **[4.0]** `df.plot` plotting API and much better error messages.
- **[4.2]** Vector similarity and sketch functions in the DataFrame API.
- **[4.2]** Faster `toPandas()` and `createDataFrame`.
- **[4.2]** Pandas API on Spark improvements.

Spark Core

Structured (JSON) Logging [4.0]

On Spark 3.5 every log line was plain text, and to get values out of it we had to write regex. Spark 4.0 adds structured logging: when `spark.log.structuredLogging.enabled=true`, every line is a JSON object with `ts`, `level`, `msg`, `logger` and a `context` map.

Plain text logs:

```
26/10/15 11:54:08 INFO SparkContext: Running Spark version 4.1.0
26/10/15 11:54:08 INFO SparkContext: Java version 21.0.9+10-LTS
```

Structured logs:

```
{"ts":"2026-10-15T11:54:27.020Z","level":"INFO","msg":"Running Spark version 4.1
{"ts":"2026-10-15T11:54:27.035Z","level":"INFO","msg":"Java version 21.0.9+10-LT
```

The nice part is that Spark can now read its own logs. PySpark ships a ready-made schema for them:

```
from pyspark.logger import SPARK_LOG_SCHEMA
logs_df = (
    spark.createDataFrame([(l,) for l in json_logs], "value STRING")
    .select(F.from_json("value", SPARK_LOG_SCHEMA).alias("log"))
    .select("log.*")
)
logs_df.where(F.col("context")["spark_version"].isNotNull()).show(truncate=False
```

Use cases: shipping logs to ELK, Loki or Datadog without writing parsing rules, and querying job logs directly with Spark SQL.

Optimizer and Shuffle Improvements [4.2]

Spark 4.2 brings several optimizer improvements that we get for free, without changing any code:

- **Subplan merging**: identical subqueries are computed once.
- **Join pushdown through Union**.
- **Faster multiple** `COUNT(DISTINCT)` in the same query.
- **Shuffle retry improvements**, order-independent shuffle checksums and general security hardening.

Web UI [4.2]

A small one, but very welcome: the Spark Web UI now has **dark mode** and the SQL plan diagrams are **searchable and zoomable**. Anyone who has tried to read a plan with 200 nodes knows why this matters.

Spark Connect

Spark Connect splits Spark into a **client** and a **server** that talk over gRPC. Our Python process no longer has a JVM, so a crash in our notebook does not kill the driver, and many notebooks can share one cluster.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.remote("sc://localhost:15002").getOrCreate()
```

- [4.0] Near-full API parity with classic mode, `spark.api.mode=connect|classic`, and a lightweight `pyspark-client` package of about **1.5 MB**.

- **[4.1] Spark ML on Connect GA**, a Connect JDBC driver, compressed plans and chunked Arrow results.
- **[4.2]** `DataFrame.toJSON`, `zipWithIndex`, a **`GetStatus` API**, faster `head()` / `take()`, and a **Connect tab in the History Server**.

We ran our whole 4.2 test notebook through Spark Connect: Delta DML and time travel, VARIANT, UDFs, polymorphic UDTFs, Python data sources, stateful streaming, the UDF profiler and ML. Everything worked. The only real gaps are **by design**, because the client has no JVM:

- `spark._jvm`
- `spark.sparkContext` (so `setLogLevel`, broadcast variables and accumulators)
- `df.rdd` and the RDD API

Structured Streaming

State Data Source Reader [4.0]

Debugging a stateful streaming query used to be painful, because the state was a black box inside the checkpoint. Spark 4.0 lets us **read the state store like a table**.

```
# RocksDB + changelog checkpointing is needed to read the state change feed
spark.conf.set("spark.sql.streaming.stateStore.providerClass",
               "org.apache.spark.sql.execution.streaming.state.RocksDBStateStore
spark.conf.set("spark.sql.streaming.stateStore.rocksdb.changelogCheckpointing.en
```

After running a simple `groupBy("user").agg(count, sum)` stream from Kafka, we can query three things straight from the checkpoint:

```
# 1. Operators, partitions and batch range
spark.read.format("state-metadata").load(checkpoint_location).show()

# 2. Current state (key -> value)
spark.read.format("statestore").load(checkpoint_location).show()
```

```
+-------+---------+------------+
|key    |value    |partition_id|
+-------+---------+------------+
|{bob}  |{9, 492} |1           |
|{alice}|{12, 610}|1           |
|{carol}|{4, 205} |1           |
+-------+---------+------------+
```

```
# 3. Change feed: what changed for each key in every batch
(spark.read.format("statestore")
    .option("readChangeFeed", True)
    .option("changeStartBatchId", 0)
    .load(checkpoint_location)
    .orderBy("batch_id")
    .show())
```

```
+--------+-----------+-------+---------+------------+
|batch_id|change_type|key    |value    |partition_id|
+--------+-----------+-------+---------+------------+
|0       |update     |{bob}  |{5, 279} |1           |
|0       |update     |{alice}|{6, 274} |1           |
|1       |update     |{alice}|{8, 412} |1           |
|2       |update     |{alice}|{10, 556}|1           |
|3       |update     |{alice}|{12, 610}|1           |
...
```

Use cases: debugging wrong streaming results, auditing state size and checking what a key held at batch N.

transformWithState [4.0, Python in 4.1]

`transformWithState` is the next generation of `mapGroupsWithState` / `flatMapGroupsWithState`. It supports **multiple named state variables, timers and TTL. [4.1]** brings a row-based Python version:

```
from pyspark.sql import Row
from pyspark.sql.streaming.stateful_processor import StatefulProcessor, Stateful

class SeenCounter(StatefulProcessor):
    def init(self, handle: StatefulProcessorHandle):
        self.seen = handle.getValueState("seen", StructType([StructField("n", Lo
    def handleInputRows(self, key, rows, timerValues):
        n = (self.seen.get()[0] if self.seen.exists() else 0) + sum(1 for _ in r
        self.seen.update((n,))
        yield Row(user=key[0], seen=n)
    def close(self): pass
query = (events.groupBy("user")
         .transformWithState(SeenCounter(), outputStructType="user STRING, seen
                             outputMode="Update", timeMode="None")
         .writeStream.format("memory").queryName("tws_out")
         .outputMode("update").option("checkpointLocation", TWS).start())
```

And because state variables have names, we can read one back directly:

```
spark.read.format("statestore").option("stateVarName", "seen").load(TWS).show()
```

Real-Time Mode [4.1, PySpark in 4.2]

Real-Time Mode brings **millisecond-level latency** to Structured Streaming, without micro-batch scheduling overhead. It came to Scala in 4.1 and to PySpark in 4.2:

```
query = (events.writeStream
         .format("console")
         .outputMode("update")
         .trigger(realTime="5 seconds")
         .start())
```

> Note: Real-Time Mode currently supports **Kafka sources only** and a list of allowed sinks. We tried it with the built-in `rate` source and got `STREAMING_REAL_TIME_MODE.INPUT_STREAM_NOT_SUPPORTED`.

Other Streaming Changes [4.2]

- **Named sources and sinks**: we can now add or reorder sources without breaking the checkpoint. This was a real pain point on 3.x.
- **Stream-stream joins in Update mode**, plus a new state format.
- **State store auto-repair and checksums**, so corrupted state is detected and repaired instead of failing the query.

Spark Declarative Pipelines [4.1]

This is probably the biggest new concept in the 4.x line. **Spark Declarative Pipelines (SDP)** is declarative ETL built into open-source Spark, in other words the open-source version of what Databricks users know as Delta Live Tables (now Lakeflow Declarative Pipelines).

Instead of writing the *how* (read, transform, write, manage checkpoints, order the jobs), we only declare the *what*: which tables and views exist and how each one is computed. Spark figures out the dependency graph, the execution order, checkpointing and retries.

```
from pyspark import pipelines as dp
```

```
@dp.materialized_view
def orders_clean():
    return spark.read.table("raw.orders").where("amount > 0")
@dp.table
def orders_stream():
    return spark.readStream.table("raw.order_events")
```

The pipeline is then run with the `spark-pipelines` CLI.

[4.2] adds **Auto CDC (SCD Type 1)**, so applying a change feed to a target table (upserts and deletes in the right order) becomes a declaration rather than a hand-written `MERGE`.

Use cases: medallion architecture (bronze → silver → gold) pipelines, where most of the code we used to write was orchestration and plumbing rather than business logic.

Deployment / Cluster Managers

- [4.0] **Mesos is removed.** If you still run Spark on Mesos, you must move to Kubernetes, YARN or standalone before upgrading.
- [4.0] The **Spark Kubernetes Operator** is released as a separate Apache project, which gives an official way to run Spark applications and clusters on Kubernetes using custom resources.
- [4.2] **Java 25 support**, so you can run Spark on the newest LTS JVM.

Closing Thoughts

Spark 4.x is not just a version bump. Spark 4.0 cleans up the foundation (modern Java, Scala 2.13, ANSI SQL by default) and turns Spark Connect into a first-class way of working. Spark 4.1 adds big new capabilities such as Declarative Pipelines, Real-Time Mode and SQL scripting. And Spark 4.2 fills

in the gaps with geospatial types, CDC, QUALIFY, Arrow UDFs by default and a long list of quality-of-life improvements.

Data Science ✓ | Data Engineering + | Spark + | Big Data +

Artificial Intelligence ✓



Written by Muaaz

261 followers · 23 following

Data Engineer with experience across batch/streaming data pipelines, Spark, Databricks, Kafka. https://www.linkedin.com/in/muaazmuzammil-ai-ml-dataengineering/

Follow